cetz-spline

Advanced Curve Geometry and Splines for CeTZ

Version: 0.1.0 **Compiler:** ≥ 0.11.0 **CeTZ:** 0.5.3 **License:** MIT

Contents

1 Overview 2
1.1 CeTZ Integration Audit 2
2 Getting Started 3
2.1 Usage 3
2.2 Named Anchors 3
3 Mathematical Concepts 4
3.1 Spline Data Representation 4
3.2 Centripetal Catmull-Rom Parameterization 4
3.3 Curvature and Normal Vectors 4
3.4 Arc-Length Quadrature and Inversion 5
4 API Reference 5
4.1 Spline Constructors 5
4.2 Analytical Evaluation 6
4.3 Arc Length & Inversion 8
4.4 Geometric Operations 8
4.5 Canvas Drawing Functions 12
Dynamic Named Anchors 12
Parameter Space (`.t()`) vs Arc-Length Space (`.s()`) 13
5 Examples Gallery 17
5.1 Aerodynamic Airfoil Profile & Curvature Comb 17
5.2 High-Speed Racetrack Corner & Banking Lanes 18
5.3 Mechanical Cam Profile & Roller Follower . . 19
5.4 Shape Morphing (Homotopy Family & Flow Field) 20
5.5 Knot Exponent Comparison (α) 21
5.6 Turbomachinery Rotor Symmetry via Rotational Transforms 22

1 Overview

`cetz-spline` provides a mathematically rigorous spline and curve geometry library built specifically for the CeTZ canvas ecosystem.

CeTZ already includes 2D/3D vector mathematics (`cetz.vector`), basic cubic Bézier evaluation (`cubic-point`, `cubic-derivative`), and de Casteljau curve splitting (`split`). `cetz-spline` directly re-uses those primitives and focuses strictly on higher-order geometric operations that CeTZ does not provide:

- **Centripetal and Chordal Catmull-Rom Parameterization** ($\alpha = 0.5, 1.0$): CeTZ only implements uniform Catmull-Rom ($\alpha = 0$), which produces self-intersections and loops around sharp corners. Centripetal parameterization mathematically prevents cusps and local self-intersections.
- **Second Derivatives and Curvature Fields**: Continuous acceleration vectors $C''(u)$, Frenet unit normal fields $\boldsymbol{N}(u)$, and signed scalar curvature $\kappa(u)$.
- **Arbitrary-Degree B-Splines**: Cox-de Boor recursion with exact conversion to cubic Bézier segments for clamped and periodic knot vectors.
- **High-Precision Arc Length**: 5-point Gauss-Legendre numerical quadrature with Newton-Raphson distance inversion ($s \mapsto t$).
- **Parallel Offset Curves**: Normal-offset curves $C(u) + d \cdot \boldsymbol{N}(u)$ with curvature-scaled tangent matching.
- **Homotopies and Deformation Vector Fields**: Morphing families $(1-t)A + tB$ and directional deformation vectors connecting curves at equal arc-length fractions.
- **CAD-Style Geometric Visualizers**: Curvature needles and envelope curves, velocity vectors, normal fields, and control polygons.

1.1 CeTZ Integration Audit

The following table summarizes the boundary between CeTZ core and `cetz-spline`:

Capability	Origin	Location	Description
Vector math	CeTZ	`cetz.vector`	Vector addition, subtraction, scaling, length, dot product.
Bézier point evaluation	CeTZ	`cetz.util.bezier.cubic-point`	Evaluates $C(u)$ at parameter $u \in [0, 1]$.
Velocity vector	CeTZ	`cetz.util.bezier.cubic-derivative`	Evaluates $C'(u) = \frac{dC}{du}$.
Curve subdivision	CeTZ	`cetz.util.bezier.split`	De Casteljau subdivision at parameter u.
Second derivative	**cetz-spline**	`cs.cubic-second-derivative`	Acceleration vector $C''(u) = \frac{d^2C}{du^2}$.
Curvature $\kappa(u)$	**cetz-spline**	`cs.cubic-curvature`	Signed scalar curvature $\frac{x'y'' - y'x''}{(x'^2 + y'^2)^{\frac{3}{2}}}$.
Frenet normal $\boldsymbol{N}(u)$	**cetz-spline**	`cs.cubic-normal`	Unit normal vector field $(-T_y, T_x)$.
Centripetal Catmull-Rom	**cetz-spline**	`cs.spline-catmull`	Yuksel (2011) $\alpha = 0.5$ knot allocation; prevents cusps.
Arbitrary B-Splines	**cetz-spline**	`cs.spline-bspline`	Cox-de Boor recursion with exact Bézier decomposition.
Arc-length quadrature	**cetz-spline**	`cs.spline-arc-length`	5-point Gauss-Legendre quadrature for exact curve length.

Arc-length inversion	**cetz-spline**	`cs.spline-t-for-distance`	Newton-Raphson solver for physical distance $s \mapsto t$.
Parallel offset curves	**cetz-spline**	`cs.offset`	Parallel normal offset with curvature-matched velocity.
Affine transforms	**cetz-spline**	`cs.rotate-spline`	Rotation, translation, directional scaling, and mirroring/reflections.
Homotopy morphing	**cetz-spline**	`cs.homotopy`	Topological loop deformation and intermediate families.
Curvature combs	**cetz-spline**	`cs.draw.draw-curvature-comb`	CAD-style curvature needles with envelope spline.

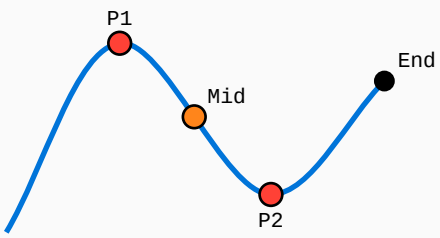
2 Getting Started

2.1 Usage

Import `cetz` and `cetz-spline` into your document:

```
#import "@preview/cetz:0.5.3"
#import "@preview/cetz-spline:0.1.0" as cs
```

Spline rendering functions are typically called inside a CeTZ canvas block:




```
let pts = ((0, 0), (1.5, 2.5), (3.5, 0.5), (5,
2))
let sp = cs.spline-catmull(pts, alpha: 0.5)

spline(sp, name: "c", stroke: 2pt + blue)
circle("c.pt-1", radius: 0.15, fill: red)
circle("c.pt-2", radius: 0.15, fill: red)
circle("c.mid", radius: 0.15, fill: orange)
circle("c.end", radius: 0.12, fill: black)
content("c.pt-1", text(size: 8pt, fill: black)
[P1], anchor: "south", padding: 7pt)
content("c.pt-2", text(size: 8pt, fill: black)
[P2], anchor: "north", padding: 7pt)
content("c.mid", text(size: 8pt, fill: black)
[Mid], anchor: "south-west", padding: 5pt)
content("c.end", text(size: 8pt, fill: black)
[End], anchor: "south-west", padding: 5pt)
```

2.2 Named Anchors

When `name: "..."` is passed to `cs.draw.spline`, the following named anchors are injected:

- `"start"`, `"end"`: curve extremities
- `"mid"`: midpoint at global parameter $t = 0.5$
- `"pt-<idx>"`: original control points (`"pt-0"`, `"pt-1"`, …)
- `"t(X%)"`: position at parameter ratio (e.g. `"t(25%)"`, `"t(50%)"`, `"t(75%)"`)
- `"s(X%)"`: position at exact physical arc-length ratio (e.g. `"s(50%)"`)
- `"50%"`, `"25%"`, `"75%"`: standard CeTZ path percentage anchors

3 Mathematical Concepts

3.1 Spline Data Representation

Every curve in `cetz-spline` is represented by an immutable dictionary:

```
#let make-spline(
  type: "catmull-rom",       // "catmull-rom" | "bspline" | "homotopy" | "offset"
  pts: (),                   // Array of original input points
  closed: false,             // bool
  meta: (:),                 // Spline parameters (alpha, degree, knots, etc.)
  segments: (),              // Array of native CeTZ cubic Bézier segments ((s, e, c1,
c2), ...)
  lut: none,                 // Cached arc-length lookup table
)
```

Each segment inside `segments` follows CeTZ's standard Bézier tuple:

$$(\boldsymbol{s}, \boldsymbol{e}, \boldsymbol{c}_1, \boldsymbol{c}_2)$$

where $\boldsymbol{s}$ is the starting point, $\boldsymbol{e}$ is the end point, and $\boldsymbol{c}_1, \boldsymbol{c}_2$ are control handles.

3.2 Centripetal Catmull-Rom Parameterization

For four consecutive interpolation points $\boldsymbol{P}_0, \boldsymbol{P}_1, \boldsymbol{P}_2, \boldsymbol{P}_3$, knot intervals are computed as:

$$t_0 = 0, \quad t_{i+1} = t_i + \|\boldsymbol{P}_{i+1} - \boldsymbol{P}_i\|^{\alpha}$$

where:

- $\alpha = 0.0$: Uniform Catmull-Rom. Susceptible to self-intersections and cusps when distance between adjacent points varies significantly.
- $\alpha = 0.5$: Centripetal Catmull-Rom (default). Proven by Yuksel et al. (2011) to be cusp-free and self-intersection-free within segments.
- $\alpha = 1.0$: Chordal Catmull-Rom. Knots are proportional to Euclidean chord length.

The Bézier control points between $\boldsymbol{P}_1$ and $\boldsymbol{P}_2$ are obtained by evaluating the endpoint derivatives:

$$\boldsymbol{B}_0 = \boldsymbol{P}_1$$

$$\boldsymbol{B}_1 = \boldsymbol{P}_1 + \frac{t_2 - t_1}{3}\left[\frac{t_2 - t_1}{t_2 - t_0}\frac{\boldsymbol{P}_1 - \boldsymbol{P}_0}{t_1 - t_0} + \frac{t_1 - t_0}{t_2 - t_0}\frac{\boldsymbol{P}_2 - \boldsymbol{P}_1}{t_2 - t_1}\right]$$

$$\boldsymbol{B}_2 = \boldsymbol{P}_2 - \frac{t_2 - t_1}{3}\left[\frac{t_3 - t_2}{t_3 - t_1}\frac{\boldsymbol{P}_2 - \boldsymbol{P}_1}{t_2 - t_1} + \frac{t_2 - t_1}{t_3 - t_1}\frac{\boldsymbol{P}_3 - \boldsymbol{P}_2}{t_3 - t_2}\right]$$

$$\boldsymbol{B}_3 = \boldsymbol{P}_2$$

3.3 Curvature and Normal Vectors

The unit tangent vector is $\boldsymbol{T}(u) = \frac{C'(u)}{\|C'(u)\|}$. The Frenet unit normal vector $\boldsymbol{N}(u)$ points 90 degrees counter-clockwise from $\boldsymbol{T}(u)$:

$$\boldsymbol{N}(u) = \left(-T_{y(u)}, T_{x(u)}\right)$$

Signed scalar curvature $\kappa(u)$ is:

$$\kappa(u) = \frac{x'(u)y''(u) - y'(u)x''(u)}{(x'(u)^2 + y'(u)^2)^{\frac{3}{2}}}$$

Curvature is positive when the curve bends to the left (counter-clockwise) and negative when bending to the right (clockwise).

3.4 Arc-Length Quadrature and Inversion

Curve speed $\|C'(t)\|$ varies along polynomial splines. Arc length between parameters a and b is evaluated using 5-point Gauss-Legendre quadrature:

$$\int_a^b \|C'(u)\|\,du \approx \frac{b-a}{2}\sum_{i=1}^{5} w_i\,\|C'\left(\frac{a+b}{2}+\frac{b-a}{2}x_i\right)\|$$

with weights and abscissae:

$$x_1 = 0, \quad w_1 = \frac{128}{225} \approx 0.568889$$

$$x_{2,3} = \pm\sqrt{\frac{5-2\sqrt{\frac{10}{7}}}{9}} \approx \pm 0.538469, \quad w_{2,3} = \frac{322+13\sqrt{70}}{900} \approx 0.478629$$

$$x_{4,5} = \pm\sqrt{\frac{5+2\sqrt{\frac{10}{7}}}{9}} \approx \pm 0.906180, \quad w_{4,5} = \frac{322-13\sqrt{70}}{900} \approx 0.236927$$

To map physical distance s back to parameter t, Newton-Raphson iteration is performed:

$$u_{k+1} = u_k - \frac{s(u_k) - s_{\text{target}}}{\|C'(u_k)\|}$$

4 API Reference

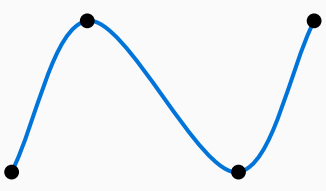
4.1 Spline Constructors

```
spline-catmull(
  points array,
  alpha float = 0.5,
  closed bool = false,
  boundary str = "clamped"
) → spline
```

Constructs a Catmull-Rom spline through a list of points using generalized knot spacing.

Parameters

- **`points`** `array`
  Array of 2D coordinates `((x0, y0), (x1, y1), ...)`.
- **`alpha`** `float`
  Knot exponent: `0.0` (uniform), `0.5` (centripetal, default), or `1.0` (chordal).
- **`closed`** `bool`
  Whether the curve is a closed periodic loop.
- **`boundary`** `str`
  Boundary condition for open curves: `"clamped"` (virtual ghost points) or `"natural"`.



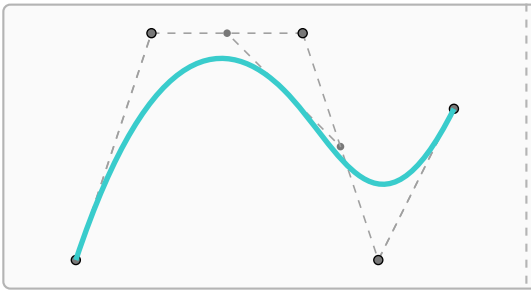
```
let pts = ((0, 0), (1, 2), (3, 0), (4, 2))
let sp = cs.spline-catmull(pts, alpha: 0.5)
spline(sp, stroke: 1.5pt + blue)
for pt in pts { circle(pt, radius: 0.08, fill:
black) }
```

```
spline-bspline(
  points array,
  degree int = 3,
  closed bool = false,
  knots array
) → spline
```

Constructs a B-spline of arbitrary degree via Cox-de Boor recursion, decomposed into cubic Bézier segments.

Parameters

- **points** `array`
  Array of control polygon vertices.
- **degree** `int`
  Polynomial degree (default: 3 for cubic B-spline).
- **closed** `bool`
  Whether the spline forms a closed periodic loop.
- **knots** `array`
  Optional custom knot vector. Defaults to clamped open-uniform knots.



```
let pts = ((0, 0), (1, 3), (3, 3), (4, 0), (5,
2))
let sp = cs.spline-bspline(pts, degree: 3)
draw-control-polygon(sp)
spline(sp, stroke: 2pt + teal)
```

4.2 Analytical Evaluation

```
spline-eval(
  spline spline,
  t float
) → point
```

Evaluates position $C(t) \in \mathbb{R}^2$ on the spline for parameter $t \in [0, 1]$.

Parameters

- **spline** `spline`
  Target spline dictionary.
- **t** `float`
  Global parameter in range $[0, 1]$.

```
spline-derivative(
  spline spline,
  t float
) → vector
```

Computes velocity vector $C'(t) = \frac{dC}{dt}$ scaled with respect to global parameter t.

```
spline-second-derivative(
  spline spline,
  t float
) → vector
```

Computes acceleration vector $C''(t) = \frac{d^2C}{dt^2}$.

```
spline-tangent(
  spline spline,
  t float
) → vector
```

Computes normalized unit tangent vector $\boldsymbol{T}(t) = \frac{C'(t)}{\|C'(t)\|}$.

```
spline-normal(
  spline spline,
  t float
) → vector
```

Computes Frenet unit normal vector $\boldsymbol{N}(t) = \left(-T_y, T_x\right)$.

```
spline-curvature(
  spline spline,
  t float
) → float
```

Computes signed scalar curvature $\kappa(t) = \frac{x'y'' - y'x''}{(x'^2 + y'^2)^{\frac{3}{2}}}$.

```
spline-normal-angle(
  spline spline,
  t float
) → angle
```

Computes the direction angle of the unit normal vector $\boldsymbol{N}(t)$ as a Typst `angle` (e.g. `90deg`, `-45deg`). Ideal for orienting perpendicular annotations, ticks, and CeTZ label anchors.

```
spline-tangent-angle(
  spline spline,
  t float
) → angle
```

Computes the direction angle of the unit tangent vector $\boldsymbol{T}(t)$ as a Typst `angle`. Useful for vehicle heading, arrow alignment, and flow orientations.

```
spline-label-anchor(
  spline spline,
  t float,
  side int = 1
) → angle
```

Computes the optimal CeTZ `content` anchor angle to position text cleanly along the curve normal without colliding with the stroke. If `side` is 1 (default), returns `spline-normal-angle + 180deg` (places text on the outer normal side). If `side` is −1, returns `spline-normal-angle` (places text on the inner side).

4.3 Arc Length & Inversion

```
spline-arc-length(
  spline spline,
  t1 float = 0.0,
  t2 float = 1.0
) → float
```

Computes arc length between parameters t_1 and t_2 via 5-point Gauss-Legendre quadrature.

```
spline-t-for-distance(
  spline spline,
  distance float
) → float
```

Finds parameter $t \in [0, 1]$ corresponding to physical distance along the curve using Newton-Raphson iteration.

```
spline-point-at-distance(
  spline spline,
  distance float
) → point
```

Evaluates point $C(s)$ at physical distance s from curve start.

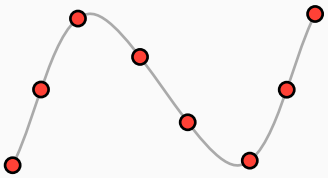
4.4 Geometric Operations

```
resample(
  spline spline,
  count int
) → array
```

Resamples the curve into `count` points spaced uniformly by physical arc length.

Parameters

- **`spline`** `spline`
  Target spline.
- **`count`** `int`
  Number of equidistant sample points (≥ 2).



```
let sp = cs.spline-catmull(((0, 0), (1, 2), (3, 0), (4, 2)))
let pts = cs.resample(sp, 8)
spline(sp, stroke: 1pt + gray)
for p in pts { circle(p, radius: 0.1, fill: red) }
```

```
homotopy(
  spline-a spline,
  spline-b spline,
  t float,
  mode str = "auto",
  samples int, auto = auto
) → spline
```

Computes intermediate morphed spline under linear homotopy $(1-t)A + tB$.

Parameters

- **spline-a** `spline`

  Base source spline ($t = 0$).
- **spline-b** `spline`

  Target destination spline ($t = 1$).
- **t** `float`

  Homotopy parameter in $[0, 1]$.
- **mode** `str`

  Interpolation strategy:
  - `"auto"` (default): uses `"segment"` if segment counts and closedness match; otherwise resamples by arc length.
  - `"segment"`: forces direct segment-by-segment Bézier control point interpolation.
  - `"arc-length"` (or `"length"`): forces uniform physical arc-length resampling before morphing, ensuring points at distance fraction s morph directly to distance fraction s on the target curve. Useful when segment lengths vary significantly between curves (e.g. $[0, 1, 8]$ to $[0, 7, 8]$) to prevent non-uniform stretching waves.
- **samples** `int, auto`

  Resample count when in `"arc-length"` mode (default: `auto`, picking $\max(4N_A, 4N_B, 32)$).

```
offset(
  spline spline,
  distance float,
  subdivide bool = true
) → spline
```

Computes a parallel offset curve displaced along the normal field $C(u) + d \cdot \boldsymbol{N}(u)$ with curvature-matched velocity.

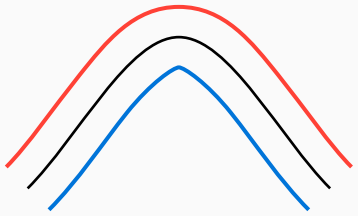
Parameters

- **spline** `spline`

  Base curve.
- **distance** `float`

  Perpendicular offset distance (positive: left, negative: right).
- **subdivide** `bool`

  Whether to split segments with high curvature variation.



```
let sp = cs.spline-catmull(((0, 0), (2, 2), (4, 0)))
let left = cs.offset(sp, 0.4)
let right = cs.offset(sp, -0.4)

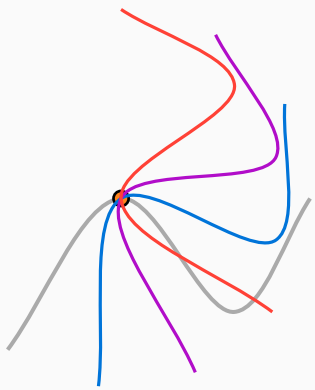
spline(left, stroke: 1.5pt + red)
spline(sp, stroke: 1pt + black)
spline(right, stroke: 1.5pt + blue)
```

```
rotate-spline(
  spline spline,
  angle angle, float,
  origin point = (0.0, 0.0)
) → spline
```

Rotates a spline rigidly by `angle` around an `origin` pivot point. Because Bézier curves are affine-invariant, all segment endpoints and control handles are transformed exactly without numerical approximation, preserving parameterization and arc lengths. Also aliased as `rotate`.

Parameters

- **`spline`** `spline`
  Target spline dictionary.
- **`angle`** `angle, float`
  Rotation angle (e.g. `45deg`, `-90deg`, or radians as float).
- **`origin`** `point`
  Pivot point of rotation (default: `(0, 0)`).



```
let sp = cs.spline-catmull(((0, 0), (1.5, 2), (3, 0.5), (4, 2)))
let piv = (1.5, 2)

spline(sp, stroke: 1.5pt + gray)
circle(piv, radius: 0.1, fill: orange)

for (i, a) in (30deg, 60deg, 90deg).enumerate()
{
  let rot = cs.rotate-spline(sp, a, origin: piv)
  let clr = (blue, purple, red).at(i)
  spline(rot, stroke: 1.2pt + clr)
}
```

```
translate-spline(
  spline spline,
  offset point
) → spline
```

Translates a spline rigidly by a 2D delta vector `offset = (dx, dy)`. Preserves arc lengths and lookup tables. Also aliased as `translate`.

Parameters

- **`spline`** `spline`
  Target spline dictionary.
- **`offset`** `point`
  Translation offset vector `(dx, dy)`.

```
scale-spline(
  spline spline,
  factor float, array,
  origin point = (0.0, 0.0),
  dir any
) → spline
```

Scales a spline relative to `origin`. Supports negative factors for reflections, as well as directional scaling along an arbitrary vector or angle `dir`. Resets the arc-length lookup table for automatic lazy rebuild. Also aliased as `scale`.

Parameters

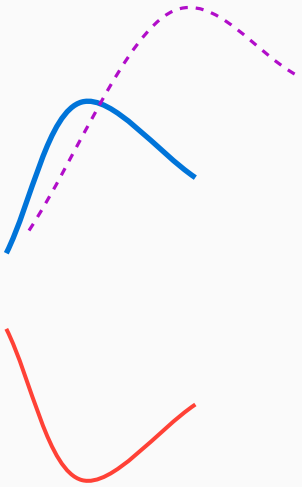
- **`spline`** `spline`
  Target spline dictionary.
- **`factor`** `float, array`
  Scale factor. If `dir` is `none`, a float scales uniformly, or `(sx, sy)` scales along axes. If `dir` is specified, a float scales along `dir`, or `(su, sv)` scales along `dir` and its normal.
- **`origin`** `point`
  Center point of scaling (default: `(0, 0)`).
- **`dir`** `str, array, angle, none`
  Optional direction vector `(dx, dy)`, angle (e.g. `45deg`), or `"x"` / `"y"`. Default: `none`.

```
mirror-spline(
  spline spline,
  origin point = (0.0, 0.0),
  axis any = "x"
) → spline
```

Mirrors (reflects) a spline across a line passing through `origin` or performs point reflection. Also aliased as `mirror`.

Parameters

- **`spline`** `spline`
  Target spline dictionary.
- **`origin`** `point`
  Point that the reflection axis/center passes through (default: `(0, 0)`).
- **`axis`** `str, array, angle, none`
  Reflection axis: `"x"` (horizontal line, flips y), `"y"` (vertical line, flips x), a direction vector `(dx, dy)`, an `angle` (e.g. `45deg`), or `none` for point reflection.



```
let sp = cs.spline-catmull(((0.5, 0.5), (1.5,
2.5), (3, 1.5)))

spline(sp, stroke: 2pt + blue)

// Mirror across x-axis
let mir-x = cs.mirror-spline(sp, axis: "x")
spline(mir-x, stroke: 1.5pt + red)

// Directional scale along (1, 1) direction
let sc-dir = cs.scale-spline(sp, 1.6, origin:
(0, 0), dir: (1, 1))
spline(sc-dir, stroke: (paint: purple,
thickness: 1.2pt, dash: "dashed"))
```

4.5 Canvas Drawing Functions

```
draw.spline(
  spline-obj any,
  name str,
  stroke stroke = 1pt + luma(0%),
  fill paint,
  close bool = false
) → element
```

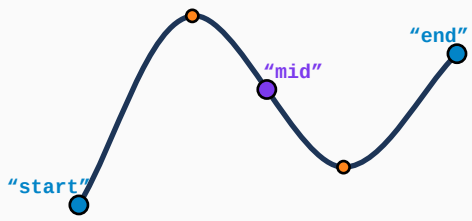
CeTZ canvas drawing element. Accepts either a spline dictionary or an array of points. When a `name` is provided, automatically injects named anchors into CeTZ's coordinate resolution system.

Dynamic Named Anchors

When a spline element is assigned a `name` (e.g. `spline(sp, name: "my-curve")`), CeTZ intercepts the coordinate lookup and dynamically evaluates requested anchor queries on the fly without precomputed limits:

Anchor Pattern	Description

`"start"`, `"end"`	First and last endpoints of the spline.
`"mid"`, `"default"`	Midpoint evaluated at parameter $t = 0.5$ (`"default"` matches `"start"`).
`"pt-0"`, `"pt-1"`, …	Original control points or segment boundary endpoints.
`"t(<val>)"`	Dynamic polynomial parameter space position $t \in [0, 1]$ or percentage $X\%$ (e.g. `"t(0.42)"`, `"t(50%)"`, `"t(33.3%)"`).
`"s(<val>)"`	Dynamic physical arc length position via Gauss-Legendre quadrature (e.g. `"s(0.5)"`, `"s(68.5%)"`, `"s(100%)"`).
`"<pct>%"`	Shorthand percentage anchor (e.g. `"25%"`, `"50%"`).
`"t-<val>"`, `"s-<val>"`	Alternate dash syntax (e.g. `"t-50"`, `"t-0_5"`, `"s-75"`, `"s-0_75"`).




```
let sp = cs.spline-catmull(((0, 0), (1.5, 2.5),
(3.5, 0.5), (5, 2)))
spline(sp, name: "c", stroke: 2pt +
rgb("#1d3557"))

// Attach labels cleanly using curve normal
angles
circle("c.start", radius: 0.12, fill:
rgb("#0284c7"))
content("c.start", text(size: 7.5pt, weight:
"bold", fill: rgb("#0284c7"))["start"], anchor:
cs.spline-label-anchor(sp, 0.0), padding: 0.15)

circle("c.mid", radius: 0.12, fill:
rgb("#7c3aed"))
content("c.mid", text(size: 7.5pt, weight:
"bold", fill: rgb("#7c3aed"))["mid"], anchor:
cs.spline-label-anchor(sp, 0.5), padding: 0.15)

circle("c.end", radius: 0.12, fill:
rgb("#0284c7"))
content("c.end", text(size: 7.5pt, weight:
"bold", fill: rgb("#0284c7"))["end"], anchor:
cs.spline-label-anchor(sp, 1.0), padding: 0.15)

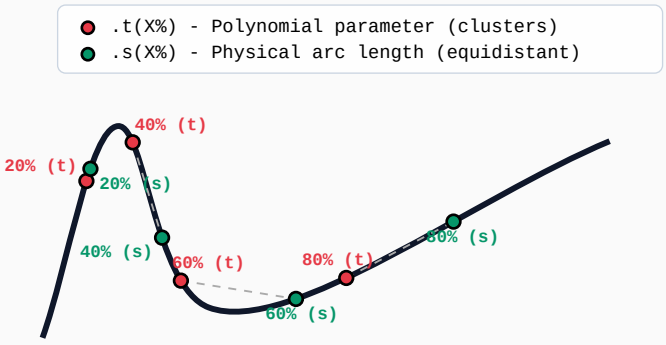
circle("c.pt-1", radius: 0.08, fill: orange)
circle("c.pt-2", radius: 0.08, fill: orange)
```

Parameter Space (`.t()`) vs Arc-Length Space (`.s()`)

A fundamental distinction in differential geometry is the difference between **polynomial parameter space** $t \in [0, 1]$ and **physical arc-length space** $s \in [0, 1]$:

- **Parameter Space (`"t(X%)"`)**: Evaluates the polynomial parameter $t = \frac{X}{100}$. Because cubic segments vary in geometric size and knot spans, equal parameter steps do **not** cover equal physical distances. Points cluster tightly where knot spans are short and spread out on long segments.
- **Arc-Length Space (`"s(X%)"`)**: Evaluates the exact physical curve distance $s = \left(\frac{X}{100}\right) \cdot L$ using 5-point Gauss-Legendre numerical quadrature with Newton-Raphson root inversion. Equal arc-length steps are physically equidistant along the curve.

The example below demonstrates this divergence on an asymmetric spline. Instead of hardcoded coordinate offsets, labels are positioned cleanly using the curve's normal angle `cs.spline-normal-angle`, which automatically directs text perpendicular to the tangent at any point:




```
let pts = ((0, 0), (1, 2.8), (2.2, 0.4), (7.5, 2.6))
let sp = cs.spline-catmull(pts, alpha: 0.5)

spline(sp, name: "asym", stroke: 2.2pt + rgb("#0f172a"))

// Compare t(p%) and s(p%) for 20%, 40%, 60%, 80%
for p in (20, 40, 60, 80) {
  let t-val = p / 100.0
  let t-anc = "asym.t(" + str(p) + "%)"
  let s-anc = "asym.s(" + str(p) + "%)"

  let n-ang-t = cs.spline-normal-angle(sp, t-val)
  let t-at-s = cs.spline-t-for-distance(sp, t-val * cs.spline-arc-length(sp))
  let n-ang-s = cs.spline-normal-angle(sp, t-at-s)

  // Connecting drift chord
  line(t-anc, s-anc, stroke: (paint: gray, thickness: 0.7pt, dash: "dashed"))

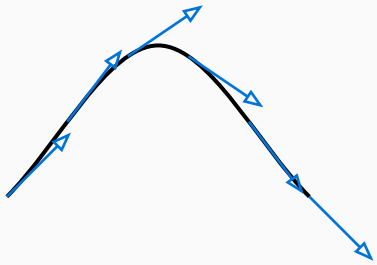
  // Parameter marker (red) positioned along outer normal
  circle(t-anc, radius: 0.09, fill: rgb("#e63946"))
  content(t-anc, text(size: 6.5pt, fill: rgb("#e63946"), weight: "bold")[#str(p)%
(t)], anchor: n-ang-t + 180deg, padding: 0.12)

  // Arc-length marker (teal) positioned along inner normal
  circle(s-anc, radius: 0.09, fill: rgb("#059669"))
  content(s-anc, text(size: 6.5pt, fill: rgb("#059669"), weight: "bold")[#str(p)%
(s)], anchor: n-ang-s, padding: 0.12)
}

// Legend
rect((0.2, 4.4), (8.2, 3.5), stroke: 0.5pt + rgb("#cbd5e1"), fill: rgb("#ffffffa"),
radius: 3pt)
circle((0.6, 4.1), radius: 0.08, fill: rgb("#e63946"))
content((0.9, 4.1), anchor: "west", text(size: 7pt)[.t(X%) - Polynomial parameter
(clusters)])
circle((0.6, 3.75), radius: 0.08, fill: rgb("#059669"))
content((0.9, 3.75), anchor: "west", text(size: 7pt)[.s(X%) - Physical arc length
(equidistant)])
```

```
draw.draw-tangents(
  spline-obj any,
  samples int = 10,
  scale float = 0.4,
  stroke stroke = 0.8pt + rgb("#0074d9")
) → element
```

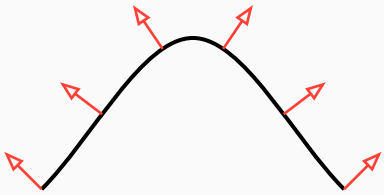
Draws directed velocity / tangent arrows along the curve.



```
let sp = cs.spline-catmull(((0, 0), (2, 2), (4, 0)))
spline(sp, stroke: 1.5pt + black)
draw-tangents(sp, samples: 6, scale: 1.2, stroke: 1pt + blue)
```

```
draw.draw-normals(
  spline-obj any,
  samples int = 10,
  scale float = 0.4,
  stroke stroke = 0.8pt + rgb("#ff4136")
) → element
```

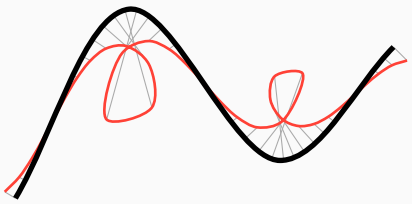
Draws perpendicular Frenet normal vectors along the curve.



```
let sp = cs.spline-catmull(((0, 0), (2, 2), (4, 0)))
spline(sp, stroke: 1.5pt + black)
draw-normals(sp, samples: 6, scale: 0.7, stroke: 1pt + red)
```

```
draw.draw-curvature-comb(
  spline-obj any,
  samples int = 40,
  scale float = 0.5,
  stroke stroke = 0.5pt + luma(66.67%),
  envelope-stroke stroke = 0.8pt + rgb("#0074d9")
) → element
```

Renders CAD-style curvature needles $\kappa(s) \cdot \boldsymbol{N}(s)$ connected by an envelope curve.



```
let sp = cs.spline-catmull(((0, 0), (1.5, 2.5),
(3.5, 0.5), (5, 2)))
draw-curvature-comb(sp, samples: 35, scale: 0.5,
stroke: 0.4pt + gray, envelope-stroke: 1pt +
red)
spline(sp, stroke: 2pt + black)
```

```
draw.draw-homotopy(
  spline-a spline,
  spline-b spline,
  steps int = 5,
  stroke-fn function,
  mode str = "auto",
  samples int, auto = auto
) → element
```

Draws a family of intermediate curves under homotopy deformation. Supports `mode: "auto"`, `"segment"`, or `"arc-length"` to control whether curves are morphed by Bézier control handles or physical arc length.

```
draw.draw-homotopy-field(
  spline-a spline,
  spline-b spline,
  samples int = 8,
  stroke stroke = 0.8pt + rgb("#0074d9"),
  mark dictionary = (end: ">"),
  mode str = "arc-length"
) → element
```

Draws directed deformation vector field arrows connecting curve A to curve B. Supports `mode: "arc-length"` (default: arrows distributed at equal physical curve distances) or `"segment"` (arrows distributed in parameter space).

5 Examples Gallery

5.1 Aerodynamic Airfoil Profile & Curvature Comb

A transonic airfoil camber profile evaluated with centripetal Catmull-Rom splines. The CAD curvature comb along the upper suction surface visualizes aerodynamic acceleration, suction peak, and smooth pressure recovery towards the trailing edge.




```
let upper-pts = ((0, 0), (0.35, 0.65), (1.2, 0.88), (2.8, 0.68), (4.5, 0.22), (5.5,
0))
let lower-pts = ((0, 0), (0.5, -0.35), (1.8, -0.38), (3.4, -0.22), (5.5, 0))

let upper = cs.spline-catmull(upper-pts, alpha: 0.5)
let lower = cs.spline-catmull(lower-pts, alpha: 0.5)

// Curvature comb on suction surface
draw-curvature-comb(
  upper,
  samples: 45,
  scale: 0.6,
  stroke: 0.4pt + luma(180),
  envelope-stroke: 1.2pt + rgb("#ef4444"),
  fill: rgb("#ef444415"),
)

// Airfoil body
spline(upper, name: "suction", stroke: 2pt + rgb("#0f172a"))
spline(lower, name: "pressure", stroke: 2pt + rgb("#0f172a"))

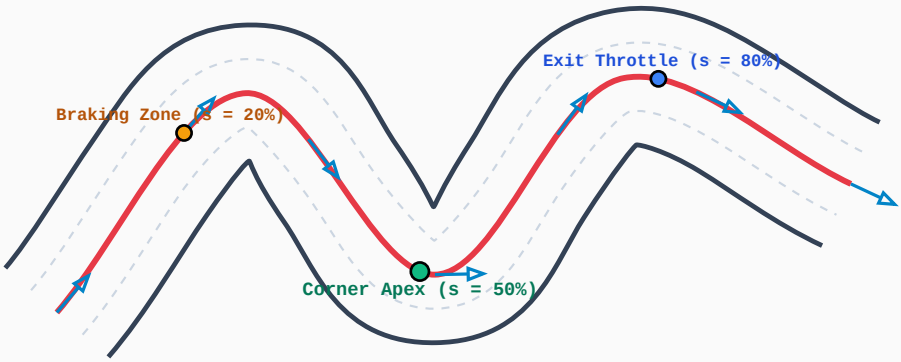
// Chord reference line
line("suction.start", "suction.end", stroke: (paint: gray, thickness: 0.6pt, dash:
"dashed"))

// Annotations
circle("suction.start", radius: 0.08, fill: rgb("#0284c7"))
content("suction.start", text(size: 7pt, weight: "bold")[Leading Edge], anchor:
45deg, padding: 0.15)

circle("suction.end", radius: 0.08, fill: rgb("#0284c7"))
content("suction.end", text(size: 7pt, weight: "bold")[Trailing Edge], anchor:
135deg, padding: 0.15)
```

5.2 High-Speed Racetrack Corner & Banking Lanes

Generating a racing line chicane with parallel track boundaries and curbs using `cs.offset`. Braking, apex, and acceleration points are positioned at exact physical arc-length distances using `"s(X%)"` anchors.




```
let track-pts = ((-4, -1.5), (-1.5, 1.4), (1, -1.0), (3.5, 1.6), (6.5, 0.2))
let racing-line = cs.spline-catmull(track-pts, alpha: 0.5)

// Track borders via parallel offset
let outer-kerb = cs.offset(racing-line, 0.9)
let inner-kerb = cs.offset(racing-line, -0.9)
let left-lane = cs.offset(racing-line, 0.45)
let right-lane = cs.offset(racing-line, -0.45)

spline(outer-kerb, stroke: 1.8pt + rgb("#334155"))
spline(left-lane, stroke: (paint: rgb("#cbd5e1"), thickness: 0.8pt, dash: "dashed"))
spline(racing-line, name: "race", stroke: 2.2pt + rgb("#e63946"))
spline(right-lane, stroke: (paint: rgb("#cbd5e1"), thickness: 0.8pt, dash: "dashed"))
spline(inner-kerb, stroke: 1.8pt + rgb("#334155"))

// Tangent vectors showing vehicle heading
draw-tangents(racing-line, samples: 7, scale: 0.7, stroke: 1.2pt + rgb("#0284c7"))

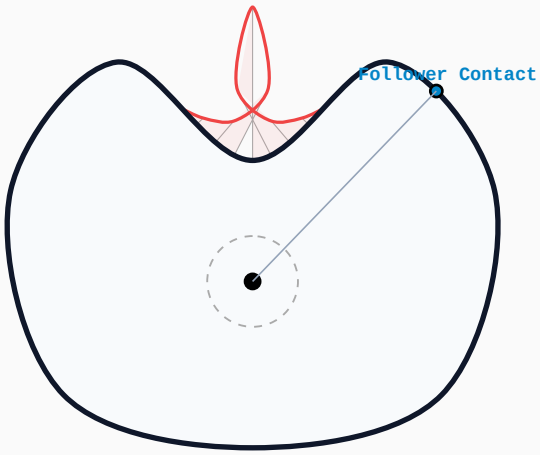
// Arc-length key positions oriented along curve normals
circle("race.s(20%)", radius: 0.1, fill: rgb("#f59e0b"))
content("race.s(20%)", text(size: 6.5pt, weight: "bold", fill: rgb("#b45309"))
[Braking Zone (s = 20%)], anchor: cs.spline-label-anchor(racing-line, 0.2), padding:
0.12)

circle("race.s(50%)", radius: 0.12, fill: rgb("#10b981"))
content("race.s(50%)", text(size: 7pt, weight: "bold", fill: rgb("#047857"))[Corner
Apex (s = 50%)], anchor: cs.spline-label-anchor(racing-line, 0.5, side: -1), padding:
0.15)

circle("race.s(80%)", radius: 0.1, fill: rgb("#3b82f6"))
content("race.s(80%)", text(size: 6.5pt, weight: "bold", fill: rgb("#1d4ed8"))[Exit
Throttle (s = 80%)], anchor: cs.spline-label-anchor(racing-line, 0.8), padding: 0.12)
```

5.3 Mechanical Cam Profile & Roller Follower

A closed C^2 cam profile evaluated with `draw-curvature-comb`. The needle spikes $\kappa(s) \cdot \boldsymbol{N}(s)$ and envelope verify acceleration smoothness, while a roller follower is positioned along the normal contact point.




```
let pts = (
  (0, 1.6), (1.8, 2.9), (3.2, 1.2), (2.5, -1.5),
  (0, -2.2), (-2.5, -1.5), (-3.2, 1.2), (-1.8, 2.9),
)
let cam = cs.spline-catmull(pts, alpha: 0.5, closed: true)

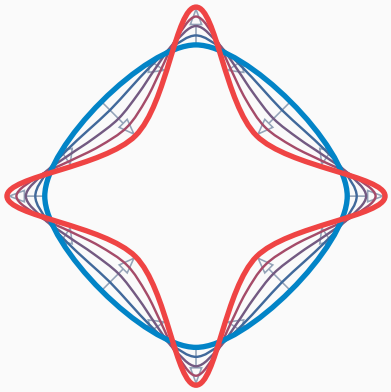
// Curvature comb
draw-curvature-comb(
  cam,
  samples: 60,
  scale: 0.8,
  stroke: 0.4pt + luma(170),
  envelope-stroke: 1.2pt + rgb("#ef4444"),
  fill: rgb("#ef444412"),
)

// Cam body & base circle
spline(cam, name: "cam-body", stroke: 2pt + rgb("#0f172a"), fill: rgb("#f8fafc"))
circle((0, 0), radius: 0.1, fill: black)
circle((0, 0), radius: 0.6, stroke: (paint: gray, thickness: 0.7pt, dash: "dashed"))

// Roller follower at peak displacement (s = 15%)
let follower-contact = "cam-body.s(15%)"
circle(follower-contact, radius: 0.08, fill: rgb("#0284c7"))
line((0, 0), follower-contact, stroke: 0.6pt + rgb("#94a3b8"))
content(follower-contact, text(size: 7pt, weight: "bold", fill: rgb("#0284c7"))
[Follower Contact], anchor: cs.spline-label-anchor(cam, 0.15), padding: 0.15)
```

5.4 Shape Morphing (Homotopy Family & Flow Field)

Continuous linear homotopy $(1-t)A + tB$ morphing between an elliptical teardrop profile and an undulating 4-lobed profile. The directional deformation vector field visualizes the displacement flux across the boundary.



```
let circle-pts = ((0, 2), (2, 0), (0, -2), (-2, 0))
let lobe-pts = ((0, 2.5), (0.8, 0.8), (2.5, 0), (0.8, -0.8), (0, -2.5), (-0.8, -0.8),
(-2.5, 0), (-0.8, 0.8))

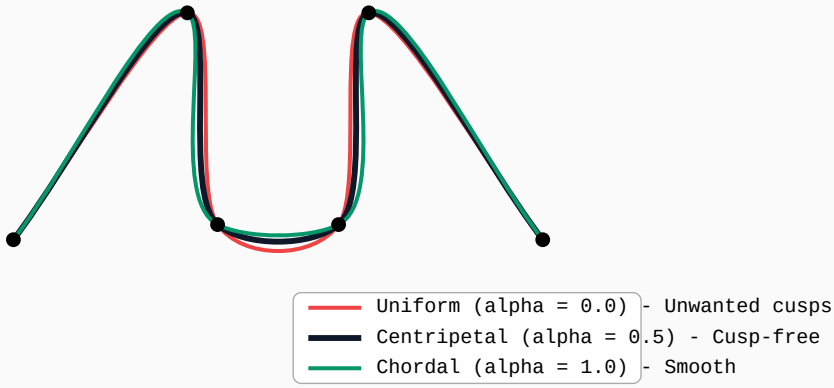
let base-shape = cs.spline-catmull(circle-pts, closed: true)
let target-shape = cs.spline-catmull(lobe-pts, closed: true)

// Deformation arrows and intermediate morph family
draw-homotopy-field(base-shape, target-shape, samples: 16, stroke: 0.6pt +
rgb("#94a3b8"))
draw-homotopy(
  base-shape,
  target-shape,
  steps: 4,
  stroke-fn: (t) => 0.9pt + rgb(int(15 + t * 200), int(120 - t * 60), int(180 - t *
100)),
)

spline(base-shape, stroke: 2pt + rgb("#0284c7"))
spline(target-shape, stroke: 2pt + rgb("#ef4444"))
```

5.5 Knot Exponent Comparison (α)

Comparison of uniform ($\alpha = 0.0$), centripetal ($\alpha = 0.5$), and chordal ($\alpha = 1.0$) Catmull-Rom parameterization over unevenly spaced control points. Centripetal parameterization mathematically eliminates the loop/cusp artifact present in the uniform curve.




```
let pts = ((-3.5, 0), (-1.2, 3), (-0.8, 0.2), (0.8, 0.2), (1.2, 3), (3.5, 0))
let sp-uni = cs.spline-catmull(pts, alpha: 0.0)
let sp-cent = cs.spline-catmull(pts, alpha: 0.5)
let sp-chord = cs.spline-catmull(pts, alpha: 1.0)

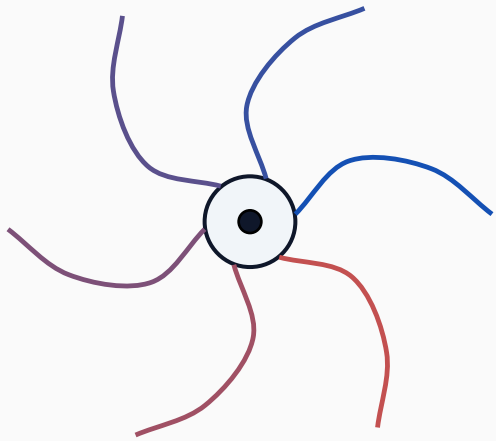
spline(sp-uni, stroke: 1.5pt + rgb("#ef4444"))
spline(sp-cent, stroke: 2.2pt + rgb("#0f172a"))
spline(sp-chord, stroke: 1.5pt + rgb("#059669"))

for p in pts { circle(p, radius: 0.08, fill: black) }

// Legend
rect((0.2, -0.7), (4.8, -1.9), stroke: 0.5pt + gray, fill: rgb("#ffffffa"), radius:
3pt)
line((0.4, -0.9), (1.1, -0.9), stroke: 1.5pt + rgb("#ef4444"))
content((1.3, -0.9), anchor: "west", text(size: 7.5pt)[Uniform (alpha = 0.0) -
Unwanted cusps])
line((0.4, -1.3), (1.1, -1.3), stroke: 2.2pt + rgb("#0f172a"))
content((1.3, -1.3), anchor: "west", text(size: 7.5pt)[Centripetal (alpha = 0.5) -
Cusp-free])
line((0.4, -1.7), (1.1, -1.7), stroke: 1.5pt + rgb("#059669"))
content((1.3, -1.7), anchor: "west", text(size: 7.5pt)[Chordal (alpha = 1.0) -
Smooth])
```

5.6 Turbomachinery Rotor Symmetry via Rotational Transforms

A single master blade profile rotated symmetrically around a central hub using `cs.rotate-spline`. Because Bézier control handles transform rigidly under affine transformations, symmetry and arc lengths are preserved exactly.



```
let blade-pts = ((0.6, 0.1), (1.3, 0.8), (2.4, 0.7), (3.2, 0.1))
let blade = cs.spline-catmull(blade-pts, alpha: 0.5)

// Central hub
circle((0, 0), radius: 0.6, stroke: 1.5pt + rgb("#0f172a"), fill: rgb("#f1f5f9"))
circle((0, 0), radius: 0.15, fill: rgb("#0f172a"))

// 6-blade rotor cascade
for i in range(6) {
  let rot-blade = cs.rotate-spline(blade, i * 60deg, origin: (0, 0))
  let clr = rgb(int(20 + i * 35), 80, int(180 - i * 20))
  spline(rot-blade, stroke: 1.8pt + clr)
}
```